

Paper Split Proposal: Detailed Handoff Document

Source Paper: "Connection-Aware Autoscaling for Stateful WebSocket Workloads in Kubernetes: Design, Empirical Evaluation, and Validation of a Custom Controller"

Authors: Abhash Behera, Aditya Hota, Suprit Kumar Naik, Ritesh Kumar Singh

Date of Proposal: June 2026

Purpose: This document is a complete handoff specification for splitting the original paper into two standalone, self-contained papers. It is intended for use by co-authors, collaborators, and downstream Claude model instances tasked with drafting either paper. No new experiments, data, or results are to be generated. All content derives strictly from the original paper.

Table of Contents

1. [Rationale for the Split](#)
 2. [Shared Infrastructure and Reuse Policy](#)
 3. [Paper 1: Full Specification](#)
 4. [Paper 2: Full Specification](#)
 5. [Cross-Paper Dependency Map](#)
 6. [Figures and Tables Assignment](#)
 7. [Equations Assignment](#)
 8. [References Assignment](#)
 9. [Abstract Drafts](#)
 10. [Key Writing Guidelines for Both Papers](#)
 11. [Risk Flags and Edge Cases](#)
-

1. Rationale for the Split

The original paper has two distinct and independently valuable scientific contributions that are currently co-located:

- **Contribution 1 (Diagnostic):** A systematic empirical characterization of *why* CPU-based HPA fails for stateful WebSocket workloads, using a progressive chain of four controlled experiments (A, B1, B2-Instrumented, B3). This contribution stands entirely on its own — the failure modes it documents are real, quantified, and actionable regardless of what solution follows.

- **Contribution 2 (Solution + Evaluation):** The design, implementation, and multi-baseline experimental validation of a custom Kubernetes controller (`StatefulAutoscaler`) that resolves the documented failure modes, including adversarial scenario analysis, protocol generalization to MQTT, and comparison against HPA+custom-metric and KEDA.

Why this split is clean:

- Paper 1 requires zero knowledge of the `StatefulAutoscaler`. Its conclusions (failure modes, root causes, quantified metrics) are self-contained.
- Paper 2 requires only a *summary* of the failure modes from Paper 1 as motivating context; it does not need to reproduce the full experimental evidence chain.
- Both papers share the same infrastructure, background, and related work, but can adapt those sections independently to their own framing.

Target venues: Both papers target the same journal tier (e.g., Springer journals in distributed systems/cloud computing). Paper 1 is a measurement/characterization paper; Paper 2 is a systems/design paper.

2. Shared Infrastructure and Reuse Policy

The following content exists in both papers but must be **independently rewritten** (not copy-pasted) to suit each paper's framing. Overlap is acceptable and expected in academic publication when two papers share an experimental infrastructure, but each paper must be self-contained.

Shared Experimental Infrastructure (appears in both papers)

- Kind cluster: 3-node (1 control-plane + 2 worker nodes), Kubernetes v1.31.6, kind v0.25.0
- Metrics Server: patched with `--metric-resolution=15s` and `--kubelet-insecure-tls`
- Base workload: custom Python asyncio WebSocket server (`ws-instrumented` image), 800 persistent client connections
- Prometheus: scrape interval 15s (baseline)
- GitHub repository: `https://github.com/AbhasBenevolentDictator/STAR` (Paper 1) and `https://github.com/MistaHolmes/ws-k8-controller` (Paper 2)

Reuse Policy

Section Type	Policy
Background (HPA architecture, KRM/PCM)	Paper 1 owns it fully; Paper 2 cites Paper 1 and provides a condensed 1-paragraph summary

Section Type	Policy
pipelines)	
Related Work	Each paper writes its own, with different emphasis (Paper 1: HPA failure literature; Paper 2: custom controller, KEDA, ML-based autoscaling literature)
Experimental infrastructure description	Both papers describe it independently but briefly; Paper 2 uses a forward reference to Paper 1 for full detail
HPA failure mode results (Experiments A, B1, B2, B3)	Paper 1 owns them fully; Paper 2 cites Paper 1 and uses a 1-paragraph motivating summary with the three key numbers (87.5% over-provisioning, 1,400 conn/s, 800→79 connections)
Controller design and algorithm	Paper 2 owns them fully; Paper 1 does not mention the <code>StatefulAutoscaler</code>
Experiments C, D, E	Paper 2 owns them fully
MQTT Extension	Paper 2 owns it fully
Adversarial Scenarios	Paper 2 owns them fully

3. Paper 1: Full Specification

Title

"CPU-Based Autoscaling is Harmful for Stateful WebSocket Workloads: An Empirical Characterization of Kubernetes HPA Failure Modes"

(Alternative title option: "When the Signal Lies: Empirical Evidence of Kubernetes HPA Failure for Persistent-Connection Workloads")

Running Title

`HPA Failure Modes for Stateful WebSocket Workloads`

Authors

Abhash Behera, Aditya Hota, Suprit Kumar Naik, Ritesh Kumar Singh (same as original)

Target Contribution Statement

Paper 1's contribution is **purely diagnostic and measurement-oriented**. It answers: *"Does CPU-*

based HPA fail for WebSocket workloads, in what ways, and how severely?" It does not propose or evaluate any solution. It motivates the need for a fundamentally different autoscaling architecture but does not deliver one.

Section-by-Section Specification

Abstract (Paper 1)

Length: ~250 words **Must contain:**

- Problem framing: CPU as invalid signal for persistent-connection workloads
 - Statement that this is a measurement/characterization paper
 - The four experiments (A, B1, B2-Instrumented, B3) as a progressive evidence chain
 - All three key quantitative findings:
 - 87.5% over-provisioning (15 replicas vs. correct 8), maxReplicas exhausted in 99 seconds
 - Peak reconnection storm: 1,400 connections/second; 51.9% connection count overshoot (1,215 vs. 800)
 - Permanent staircase connection destruction: 800→79 active connections with zero recovery
 - Conclusion: CPU-based HPA is structurally invalid (not merely suboptimal) for this workload class; the failure cannot be remediated by parameter tuning
 - Forward pointer: motivates the need for connection-aware autoscaling (without describing any solution)
 - Keywords: Kubernetes, Horizontal Pod Autoscaler, WebSocket, Stateful Workloads, Reconnection Storm, Prometheus, Connection-Aware Autoscaling, Measurement Study
-

Section 1: Introduction (Paper 1)

Length: ~600–700 words **Content:**

1. **Opening context:** Kubernetes HPA as the default autoscaling primitive; its proportional control law; its design assumption (CPU encodes demand). Cite: burns2016borg, verma2015large, k8s_hpa_docs, nguyen2020hpa.
2. **The fundamental shift:** WebSocket, IoT telemetry, collaborative editing, financial streams, multiplayer gaming — persistent-connection workloads where the unit of capacity is an *active network connection*, not a CPU cycle. Cite: websocket_rfc.
3. **The mismatch:** Define the mismatch explicitly:

- A server with 800 idle WebSocket sessions may show near-zero CPU
 - Pod termination is immediately and irrecoverably destructive to all hosted connections
 - HPA cannot distinguish "idle but connected" from "genuinely unloaded"
 - Define "reconnection storm" here as a term of art
4. **Gap in prior work:** Cite dickel2019stateful as the closest related work; state that it motivates the problem but provides neither quantitative HPA failure characterization nor a measured reconnection storm rate.
5. **Contributions of Paper 1** (numbered list):
- **Contribution 1:** Progressive four-experiment evidence chain (A → B1 → B2-Instrumented → B3) characterizing HPA failure from over-provisioning → reconnection storm measurement → permanent connection destruction
 - **Contribution 2:** First empirical measurement of reconnection storm magnitude as a direct causal consequence of HPA scale-down (peak 1,400 conn/s)
 - **Contribution 3:** Causal proof that idle-connection scale-down is architecturally inevitable with CPU-based HPA — not a configuration defect
 - **Contribution 4:** Establishing that the failure cannot be remediated by HPA parameter tuning alone (stabilization window semantics argument)
 - **Contribution 5:** Quantified failure metrics at connection-second granularity, enabling comparison baselines for future autoscaling research
6. **Paper structure:** One sentence per section.
-

Section 2: Related Work (Paper 1)

Length: ~500 words **Subsections and emphasis:**

1. Kubernetes HPA Architecture and Behavior

- Nguyen et al. (nguyen2020hpa): HPA feedback loop characterization, KRM vs. PCM pipelines, scrape interval effects. State that their framework did not evaluate stateful workloads.
- Casalicchio & Perciballi (casalicchio2019auto): metric choice (relative vs. absolute) and scaling stability for HTTP workloads. Note their evaluation is confined to CPU/memory for stateless workloads.
- lorido2014auto: general HPA behavior survey.

2. Custom and Application-Level Metrics for HPA

- Prometheus Adapter (prometheus_adapter): enables HPA to scale on application-defined metrics

- Qu et al. (qu2018auto): HTTP-rate-based autoscaling
- Rossi et al. (rossi2020horizontal): hybrid CPU-and-traffic strategies
- **Key gap to state:** All these presuppose the scaling metric is *stateless across pod boundaries* — an assumption violated by connection-holding protocols. Note that even with a correct metric, HPA's stabilization semantics may still be insufficient (foreshadow Paper 2's finding, without detail).

3. Stateful Workloads and Protocol-Aware Autoscaling

- Dickel et al. (dicke12019stateful): identifies stateful autoscaling as a challenge for IoT/WebSocket/MQTT; proposes monitoring active connections via Prometheus. **State clearly:** they provide no quantitative HPA failure characterization and no controller-level solution. Paper 1 fills the characterization gap; Paper 2 (cite as companion paper) fills the solution gap.

4. Graceful Connection Draining

- Note that preStop hooks and terminationGracePeriodSeconds are structurally insufficient for long-lived WebSocket connections. The 30-second limbo window between SIGTERM and SIGKILL is explicitly characterized in this paper's experiments.

5. Research Gap

- Summarize: no prior work has (i) empirically quantified specific HPA failure modes for WebSocket at connection-second granularity, (ii) measured reconnection storm magnitude causally, or (iii) established that the failure is architectural rather than configurational.

Section 3: Background — Kubernetes Autoscaling Pipelines (Paper 1)

Length: ~500 words + figures **This section is IDENTICAL in scope to the original paper's Section 3. Paper 1 owns this section fully.**

Content:

1. Subsection 3.1: KRM Pipeline

- cAdvisor → kubelet → Metrics Server → metrics.k8s.io API → HPA
- Baseline experiment: CPU-intensive HTTP workload, square-wave demand (100s HIGH at 50 req/s, 100s LOW at 2 req/s), 4-node Kind cluster
- Four behavioral characteristics: staircase scaling, CPU overshoot, actuation delay, stable convergence
- **Figures:** Figure 1 (HPA architecture, reproduced from nguyen2020hpa), Figure 2 (KRM CPU over time), Figure 3 (desired vs. current replicas)

2. Subsection 3.2: PCM Pipeline

- Application → Prometheus scrape → Prometheus Adapter → Custom Metrics API → HPA
- Three configurations: PCM-CPU, PCM-H, PCM-CH
- Three scrape intervals: 15s, 30s, 60s
- Key findings: 60s→30s improvement is material; 30s→15s is marginal; PCM-H leads but over-provisions; PCM-CH reduces oscillation
- **Figure:** Figure 4 (PCM scrape interval comparison)

3. Subsection 3.3: The HPA Control Architecture

- Present Equation 1 (HPA proportional control law):
$$\text{desiredReplicas} = \frac{\text{currentReplicas} \times (\text{currentMetricValue} / \text{targetMetricValue})}{1}$$
 - State the implicit assumption: currentMetricValue faithfully represents workload intensity
 - State where this assumption breaks: persistent-connection workloads
-

Section 4: Problem — Empirical Evidence of HPA Failure for Stateful WebSocket Workloads (Paper 1)

Length: ~1,800–2,000 words + figures **This is the core section of Paper 1. Maps directly to Section 4 of the original.**

Opening paragraph: Describe the shared infrastructure (Kind cluster, Python asyncio server, 800 clients, HPA with targetCPUUtilizationPercentage: 60). State the progressive chain structure. Include the instrumented server definition (Prometheus gauge `active_connections`, counter `new_connections_total`, `/drain` endpoint).

Subsection 4.1: Experiment A — HPA Baseline Under Monotonic Correlated Load

- Hypothesis, setup, results — verbatim structure from original
- Setup: 800 clients ramp over 300s, CPU_WORK=1 (CPU ∝ connections), minReplicas=2, maxReplicas=10, targetCPU=60%
- Results: Clean 2→5 replica staircase, 300s stabilization window, controlled descent. No oscillation, no disruption.
- **Figure:** Replica count timeline (Experiment A)
- **Key Result Box:** "CPU-based HPA functions correctly only under the idealized condition of perfect CPU-to-connection proportionality."

Subsection 4.2: Experiment B1 — HPA Under Cyclic Load

- Hypothesis, setup, results — verbatim structure from original
- Setup: HIGH=800 connections with pinging (60s), LOW=800 connections idle, no pinging (30s). maxReplicas=15, stabilizationWindowSeconds=300 (default).
- Results: maxReplicas=15 reached in 99 seconds. 87.5% over-provisioning (vs. correct 8 replicas). Recovery requires ~650 seconds. Mechanism: 90s cycle period < 300s stabilization window → ratcheting to ceiling.
- **Figure:** Replica count timeline showing saturation at 15 (Experiment B1)
- **Key Result Box:** "87.5% over-provisioning rate and disproportionate recovery time (650s) represent significant resource efficiency penalties."

Methodological Note (before B2): Include the B2-Extended LOW pilot note exactly as in the original — explains why the non-instrumented run is not reported separately, and why instrumentation was added.

Subsection 4.3: Experiment B2-Instrumented — Quantifying Reconnection Storm Intensity

- Hypothesis, setup, results — verbatim structure from original
- Setup: Server instrumented with `active_connections` gauge + `new_connections_total` counter. Prometheus 15s scrape. stabilizationWindowSeconds=0 for scale-up, 60s for scale-down. HIGH=60s (pinging, CPU_WORK=1), LOW=90s (idle, no pinging). Five cycles.
- Results:
 - Per-cycle storm rates: Cycle 1: 1,400.9 conn/s; Cycle 2: 1,298.3; Cycle 3: 1,399.5; Cycle 4: 1,251.8; Cycle 5: degraded
 - Peak: **1,400 connections/second**
 - Connection overshoot: **1,215 connections** (51.9% above 800-client target)
 - Mechanism: SIGKILL → TCP RST → simultaneous reconnect → TIME_WAIT overlap → transient gauge inflation
- **Figures:** reconnections.png, connections.png, scaling_activity.png (Experiment B2)
- **Key Result Box:** "HPA scale-down events are inherently destructive to WebSocket client sessions... At 1,400 conn/s, such storms represent a severe degradation event for latency-sensitive applications."

Subsection 4.4: Experiment B3 — Permanent Connection Loss Under Non-Reconnecting Clients

- Hypothesis, setup, results — verbatim structure from original
- Setup: reconnect=false. stabilizationWindowSeconds=0 (scale-up), 60s (scale-down). CPU_WORK=1. IDLE_MAX_DURATION=240s.
- Include the "**Pod termination mechanics and the 30-second limbo**" paragraph in full — this is critical background for understanding the figures.

- Results: Staircase destruction: 800→744→697→445→79. Zero recovery. Each step causally linked to an HPA scale-down decision.
- **Figures:** connections.png (staircase), combined.png (causal proof: CPU drops → replicas drop → connections destroyed) (Experiment B3)
- **Key Result Box:** "The exclusive reliance of HPA on CPU as a scaling signal causes the autoscaler to be structurally antagonistic to idle-but-connected stateful workloads. This finding demonstrates that the problem cannot be remediated by parameter tuning; it requires a fundamentally different scaling architecture."

Subsection 4.5: Summary of the Evidence Chain

- **Table 1:** Progressive evidence chain summary (Experiment → Condition → HPA Behavior → Key Quantitative Finding) — exactly as in original Table 1.
-

Section 5: Discussion (Paper 1)

Length: ~500 words **This maps to a subset of the original Discussion section. Include only the subsections relevant to Paper 1's scope.**

Subsection 5.1: CPU as an Invalid Scaling Signal for Stateful Workloads

- Reproduce argument from original: CPU is not merely suboptimal — it is structurally invalid.
- The signal drops precisely *because* connections are idle → HPA destroys the connections → circular destruction.
- Experiment C result (CPU_WORK=0, correct scaling) is cited here as evidence from the companion paper (Paper 2) that a correct signal on the same control structure produces correct behavior.
- Key phrasing: "a perfect proportional controller on the wrong signal produces worse outcomes than a simpler controller on the correct signal."

Subsection 5.2: The Stabilization Window Semantic Mismatch

- Explain *why* tuning HPA's stabilization window cannot fix the problem.
- HPA window buffers metric-derived replica recommendations. With persistent near-zero CPU, every window entry recommends minReplicas → stabilized output = minReplicas.
- The correct window must buffer *connection-derived desired counts* — retaining the memory that 8 replicas were required even when connections transiently drop to zero.
- Conclude: "This is not a tuning problem; it is a semantic mismatch between what the window stabilizes."

- Forward pointer: "The companion paper (Paper 2) implements and evaluates a controller with this corrected stabilization semantic."

Subsection 5.3: The 30-Second Limbo Window as a Design Constraint

- Formalize the SIGTERM→SIGKILL sequence (30-second TCP RST propagation lag) as an explicit design constraint for any autoscaling solution.
 - Note: preStop hooks and terminationGracePeriodSeconds are insufficient; the correct mitigation is scale-down *suppression*, not graceful drain duration extension.
-

Section 6: Limitations (Paper 1)

Length: ~300 words **Include only limitations that are relevant to Paper 1's scope (the measurement study).** Do NOT include limitations that belong to the `StatefulAutoscaler` design.

Applicable limitations from the original (adapt wording):

1. **Evaluation environment:** Single-machine Kind cluster; results are directionally valid but absolute timing values may differ on production infrastructure.
 2. **Workload generalizability:** Synthetic linear ramp and deterministic silence gaps; real-world traffic is burstier with variable session durations.
 3. **Instrumentation dependency:** B2-Instrumented results depend on Prometheus metric freshness at 15s scrape interval; higher intervals would affect storm rate measurement precision.
 4. **No non-reconnecting client variety:** B3 uses a single reconnect=false regime; real deployments have a mix of reconnecting and non-reconnecting clients; the staircase destruction rate would differ.
 5. **Single-run B3:** Experiment B3 was not replicated across five runs (unlike Experiment C in Paper 2). Results are reported from a single representative run.
-

Section 7: Conclusion (Paper 1)

Length: ~250 words **Must contain:**

- Restate the paper's scope: empirical characterization of CPU-based HPA failure modes for stateful WebSocket workloads.
- Summarize the three failure modes with their key numbers:
 - Over-provisioning: 87.5% (15 vs. 8 replicas), maxReplicas in 99 seconds

- Reconnection storm: 1,400 conn/s peak, 51.9% overshoot
- Permanent destruction: 800→79 connections, zero recovery
- State the root cause: CPU as a scaling signal is structurally invalid for persistent-connection workloads. The failure is architectural, not configurational.
- State the implication: any autoscaling solution for stateful WebSocket workloads must replace the scaling signal AND address scale-down lifecycle semantics.
- Forward pointer: "The companion paper [Paper 2 citation] presents the design, implementation, and experimental validation of the `StatefulAutoscaler`, a custom Kubernetes controller that addresses all three documented failure modes."

Data Availability (Paper 1)

Point to: `https://github.com/AbhasBenevolentDictator/STAR` Describe: experiment scripts A, B1, B2, B3; load generator; cluster config; raw results; analysis scripts.

Paper 1 — Section Order Summary

#	Section Title	~Word Count
—	Abstract	250
1	Introduction	650
2	Related Work	500
3	Background: Kubernetes Autoscaling Pipelines	500
4	Problem: Empirical Evidence of HPA Failure	1,900
5	Discussion	500
6	Limitations	300
7	Conclusion	250
—	Data Availability + Acknowledgements	100
Total		~4,950

4. Paper 2: Full Specification

Title

"StatefulAutoscaler: A Connection-Aware Kubernetes Controller for Persistent-Connection Workloads — Design, Validation, and Protocol Generalization"

(Alternative: *"Connection-Aware Autoscaling with Sliding-Window Stabilization: Design and Experimental Validation of a Custom Kubernetes Controller for WebSocket and MQTT Workloads"*)

Running Title

StatefulAutoscaler: Connection-Aware Kubernetes Autoscaling

Authors

Abhash Behera, Aditya Hota, Suprit Kumar Naik, Ritesh Kumar Singh

Target Contribution Statement

Paper 2's contribution is **design, implementation, and multi-baseline experimental validation**. It answers: *"How should a connection-aware autoscaler be designed, and does it empirically eliminate the failure modes of CPU-based HPA under WebSocket and MQTT workloads?"* It takes the failure characterization from Paper 1 as established prior work and proceeds directly to the solution.

Section-by-Section Specification

Abstract (Paper 2)

Length: ~300 words **Must contain:**

- Motivation (brief, 2-3 sentences): cite Paper 1 as establishing that CPU-based HPA fails for stateful WebSocket workloads in three specific ways (87.5% over-provisioning, 1,400 conn/s reconnection storms, permanent staircase connection destruction).
- The `StatefulAutoscaler`: what it is (custom CRD-based Kubernetes controller, Kubebuilder SDK), what it scales on (`sum(active_connections)` from Prometheus), and its key mechanism (sliding-window stabilization via `scaleDownCooldownSeconds`).
- Experiment C results (five runs, runs 2-5): exact replica targeting (8 replicas = $\lceil 800/100 \rceil$, 0% waste), zero connection loss, 8 warm pods through 90-second DROP 1 gap, scale-down reaction $119 \pm 2s$, pod-seconds $4,280 \pm 67$.

- Baseline comparison summary: HPA+custom metric (Experiment D): preserves pods through DROP 1 but $2.4\times$ slower scale-up (54s vs. 22s) and $2.6\times$ longer final scale-down (313s vs. 119s). KEDA (Experiment E): matches pod-preservation but lacks per-cycle step-rate bounding.
 - Adversarial envelope: three failure scenarios characterized (metric staleness, instantaneous spike, Prometheus outage) with explicit boundary conditions.
 - MQTT generalization: identical controller binary, zero source-code changes, 44% faster scale-up than HPA (38s vs. 68s), 0 sessions destroyed vs. ~ 300 in MQTT-A.
 - Keywords: Kubernetes, Custom Controller, StatefulAutoscaler, WebSocket, MQTT, Connection-Aware Autoscaling, Prometheus, Sliding-Window Stabilization, KEDA, Kubebuilder
-

Section 1: Introduction (Paper 2)

Length: ~ 600 words **Content:**

1. **Opening:** The failure of CPU-based HPA for stateful workloads is established — cite Paper 1 here. Three failure modes quantified: over-provisioning (87.5%), reconnection storms (1,400 conn/s), permanent connection destruction (800 $\rightarrow$ 79). State that these failure modes are architectural and cannot be resolved by HPA parameter tuning alone.
2. **The design gap:** No existing Kubernetes-native mechanism provides both (a) a connection-count scaling signal AND (b) connection-context-aware stabilization semantics. The Prometheus Adapter can supply the signal but cannot change HPA's stabilization semantics. KEDA provides event-driven scaling but delegates termination to HPA without per-cycle step bounding.
3. **This paper's approach:** We implement the `StatefulAutoscaler` — a custom Kubernetes controller built with Kubebuilder — that owns both the metric selection and the scale-down lifecycle. Describe the two design levers: (a) connection-density signal, (b) sliding-window stabilization.
4. **Contributions of Paper 2** (numbered list):
 - **Contribution 1:** Complete design and implementation of the `StatefulAutoscaler` CRD, reconciliation algorithm, and stabilization window mechanism.
 - **Contribution 2:** Experimental validation under a two-cycle restart simulation (Experiment C, five runs) demonstrating exact replica targeting, zero connection loss, and pod preservation through a 90-second disconnection gap.
 - **Contribution 3:** Comparative baseline evaluation against HPA+custom metric (Experiment D) and KEDA (Experiment E), five runs each, identifying the precise differentiating characteristics of each approach.

- **Contribution 4:** Adversarial failure scenario characterization (metric staleness, instantaneous spike, Prometheus outage) establishing the operational envelope of the `StatefulAutoscaler`.
- **Contribution 5:** Protocol generalization validation on MQTT, demonstrating that the controller binary operates without source-code changes and resolves MQTT-A's HPA failure modes.

5. **Paper structure:** One sentence per section.

Section 2: Related Work (Paper 2)

Length: ~550 words **Different emphasis from Paper 1's related work — focus on solution space, not problem space.**

Subsections:

1. Kubernetes HPA with Custom Metrics

- Prometheus Adapter (`prometheus_adapter`): enables application-defined scaling signals
- Qu et al. (`qu2018auto`), Rossi et al. (`rossi2020horizontal`): HTTP-rate-based and hybrid strategies
- **Key gap:** metric choice alone is insufficient; HPA's stabilization semantics remain CPU-oriented even with correct metrics (empirically demonstrated in Experiment D)

2. Event-Driven Autoscaling: KEDA

- KEDA (`keda_docs`): extends HPA via `minReplicas` management; `cooldownPeriod` as the hold mechanism
- Explain the delegation model: KEDA raises/lowers `minReplicas`; HPA executes termination
- **Two structural limitations for stateful workloads:** (a) no `maxScaleDownStep` bounding; (b) no awareness of OS-level TCP RST propagation window
- State that Experiment E directly tests KEDA with a matched `cooldownPeriod=120s`

3. Stateful Workloads and Protocol-Aware Autoscaling

- Dickel et al. (`dickel2019stateful`): motivates protocol-aware scaling; does not implement or validate a controller. Paper 1 (cite as companion) provides quantitative characterization. This paper provides the implementation and validation.

4. Machine Learning and Predictive Autoscaling

- Rossi et al. (`rossi2020horizontal`), Imdoukh et al. (`imdoukh2020machine`): RL approaches

- Toka et al. (toka2021machine), Zhong et al. (zhong2020machine): ML-based prediction
- Note: proactive capability addresses provisioning lag but not connection-state destruction during scale-down, which persists regardless of when scaling decisions are made.

5. Custom Kubernetes Controllers

- Kubebuilder (kubebuilder_docs): SDK for controller-runtime-based operators
- Note the CRD pattern as the standard extension mechanism; contrast with HPA's fixed signal set.

6. MQTT Autoscaling

- Soni et al. (soni2017mqtt): MQTT protocol characteristics relevant to autoscaling
- MQTT-specific limitations of CPU-based HPA (session pinning, low-CPU idle sessions) — cite MQTT-A from this paper.

Section 3: Background (Paper 2)

Length: ~300 words (condensed relative to Paper 1) **Purpose:** Provide enough context for a reader who has not read Paper 1.

Content:

- Brief description of HPA proportional control law (Equation 1, same as Paper 1). State that this is characterized in detail in Paper 1 (cite).
- One paragraph on why CPU fails for WebSocket (the mismatch): idle connections → near-zero CPU → HPA scales down → connection destruction. Cite Paper 1 for full characterization and quantification.
- One paragraph on the experimental infrastructure (Kind cluster, Python asyncio server, 800 clients, Prometheus).
- The HPA architecture figure (Figure 1, reproduced from nguyen2020hpa) — include it here as background, with a note that Paper 1 provides full pipeline characterization.
- **Do NOT reproduce the KRM/PCM baseline experiments.** One sentence: "KRM and PCM pipeline baselines for stateless workloads, establishing HPA's correct behavior under CPU-proportional load, are documented in Paper 1 (cite)."

Section 4: The `StatefulAutoscaler` Controller — Design and Implementation (Paper 2)

Length: ~1,000 words + algorithm + figures **This is the primary design section. Maps directly to Section 5 of the original.**

Subsection 4.1: Design Principles Three architectural principles (verbatim from original, with minor adaptation):

1. Use the correct scaling signal: $\text{sum}(\text{active_connections})$ from Prometheus. CPU never consulted.
2. Compute replica counts from connection density: Equation 2 —
$$\text{desiredReplicas} = \frac{\text{sum}(\text{active_connections})}{T}$$
3. Stabilize scale-down decisions using connection-context history: sliding-window maximum over preceding N seconds.

Subsection 4.2: Custom Resource Definition

- Table 1: CRD specification fields (targetRef, minReplicas, maxReplicas, targetConnsPerPod, maxScaleUpStep, maxScaleDownStep, scaleDownCooldownSec) with operational purpose for each.

Subsection 4.3: System Architecture Overview

- Describe the five-stage data and control flow (clients → Service → pods → Prometheus scrape → controller query → spec.replicas patch).
- **Figure:** StatefulAutoscaler system architecture and CRD example (Figure from original Section 5.3).
- Emphasize: CPU explicitly excluded from the control path.

Subsection 4.4: Reconciliation Loop Design

- **Algorithm 1:** $\text{StatefulAutoscaler}$ Reconciliation Logic — reproduce exactly as in original (20-line pseudocode with Require/Ensure, numbered lines, comments).
- Walk through each phase: Prometheus query → clamp → window append → window evict → stabilized = max(W) → three-way comparison (scale-up / scale-down / hold) → PatchDeployment → requeue 5s.
- **Figure:** UML activity diagram of the reconciliation loop — reproduce from original.

Subsection 4.5: The Stabilization Window — Mechanism and Correctness

- Without stabilization: 0 connections → clamped to minReplicas=2 → scale-down → re-provisioning lag on reconnect.
- With 120s window: 0 connections appended, but window contains desired=8 entries from preceding 120s → stabilized = max(W) = 8 → all 8 pods held warm.

- **Critical distinction from HPA's built-in window:** HPA stabilizes metric-signal recommendations (near-zero CPU → every entry recommends minReplicas → output = minReplicas). This controller stabilizes connection-derived desired counts (retains high-water mark). Cannot be replicated by tuning HPA window duration.

Subsection 4.6: Controller Design and Stability Properties

- Feedback loop framing: Observe → Compare → Act structure (mirrors HPA but with correct signal and hysteresis)
 - Hysteresis mechanism: scaleDownCooldownSeconds as dead zone preventing oscillation
 - Convergence bound: Equation 3 — $\lceil (current - desired) / maxScaleDownStep \rceil$ reconciliation cycles. Example: 8→2 with step=2 → 3 cycles ≈ 45 seconds.
 - Parameter selection and sensitivity: why targetConnsPerPod=100, maxScaleDownStep=2, scaleDownCooldownSeconds=120 were chosen; sensitivity analysis.
 - Delay margin: scrape lag (15s) + reconcile period (15s) = 30s max observation lag << 120s cooldown → lag fully absorbed.
 - Oscillation condition: why oscillation cannot occur (cooldown gate + step damping = dead-zone + damping pair).
-

Section 5: Experimental Evaluation — Experiment C (Paper 2)

Length: ~1,100 words + figures + tables **Maps directly to Section 6 of the original.**

Subsection 5.1: Experimental Configuration

- Table 2: Experiment C vs. B3 parametric comparison (server image, CPU_WORK, client count, autoscaler, scale-down guard, scaling signal). Highlight differences in bold.
- StatefulAutoscaler config: targetConnsPerPod=100, maxScaleUpStep=3, maxScaleDownStep=2, minReplicas=2, maxReplicas=15, scaleDownCooldownSeconds=120.
- Metric definitions: pod-seconds, scale-up reaction time (defined precisely as in original).
- CPU_WORK=0 rationale: demonstrates controller operates with zero CPU activity — the exact worst case for any CPU-based autoscaler.

Subsection 5.2: Load Pattern — Two-Cycle Restorm Simulation

- Four phases with timing: CYCLE 1 (0–150s, 800 clients), DROP 1 (150–240s, 0 connections, critical test), CYCLE 2 (240–390s, fresh 800 clients), FINAL DROP (390–570s, permanent).
- Include the terminology box: CYCLE, DROP, DROP 1, FINAL DROP, reconnection storm, two-cycle restorm simulation.

Subsection 5.3: CYCLE 1 — Connection-Driven Scale-Up

- Monotonic scale trajectory: 2→3→4→5→6→7→8 over ~60 seconds.
- CPU consumption: below 176 millicores throughout. Confirms zero CPU dependency.
- **Figure:** Active connection count over full timeline (Experiment C connections.png).

Subsection 5.4: DROP 1 — Empirical Validation of the Stabilization Window

- At $t \approx 200$ s: all clients deleted. Connections → 0 within 15s.
- Replica count: **perfectly flat at 8 for the full 90-second gap.**
- Show reconciliation trace (bulleted: C=0 → rawDesired=0 → clamped=2 → W contains desired=8 → stabilized=8 → no patch invoked).
- **Figure:** Replica count showing the "flat bridge" (Experiment C replicas.png).
- Contrast explicitly with Experiment B3: "at the equivalent elapsed time in Experiment B3, the HPA had already permanently destroyed 56 client connections via pod termination."

Subsection 5.5: CYCLE 2 — Restorm-Free Reconnection

- Fresh 800 clients: immediate connection due to pre-warmed pods. Peak: 854 connections. 8 replicas held stably.
- Compare to B2-Instrumented: 1,400 conn/s storm + 51.9% overshoot → Experiment C: zero measurable reconnection storm activity.
- **Window boundary transient dip:** Briefly describe the observed 5-6 replica dip during $t \approx 270-310$ s. Mechanism: CYCLE 1 window entries age out during DROP 1 before CYCLE 2 entries build up. Recovery within 10-15s with no connection loss. Cross-reference Limitations (item 7).

Subsection 5.6: FINAL DROP — Controlled and Graceful Scale-Down

- 120s wait for window expiry → graceful descent: 9→8→6→4→2 over ~30s.
- Zero connections exist at scale-down time → zero connections lost.

Subsection 5.7: Combined View and Definitive Comparison

- **Figure:** Combined signal view (connections, CPU, replicas) — Experiment C combined.png.
- Describe the structural inversion vs. B3: in B3, CPU drives replicas to destroy connections; in C, connections drive replicas while CPU is irrelevant.
- **Table 3:** Head-to-head quantitative comparison (HPA/B3 vs. StatefulAutoscaler/C) — reproduce Table 2 from original exactly (9 rows: primary scaling signal, replicas for 800

connections, connections lost, pods through 90s gap, reconnection storm, time to serve CYCLE 2, CPU dependency, total scaling events, peak CPU).

Subsection 5.8: Reproducibility — Five-Run Statistical Validation

- Run 1 outlier: explain the persistent-replica artefact; exclude from statistical summary.
 - Runs 2–5 statistics:
 - Scale-up reaction: 22 ± 6 s (median 21s; range 16–30s)
 - Scale-down reaction: 119 ± 2 s (median 118s; range 117–122s)
 - Pod-seconds: $4,280 \pm 67$
 - Peak replicas: 9–10 (mild overshoot from discrete-step latency)
 - Peak connections: 813–917
 - All four runs: 8 warm replicas through DROP 1, zero connection loss
 - Statistical confidence note: narrow variance ($\sigma=2$ s, $\sigma=67$) consistent with $p<0.05$ under one-sample t-test against HPA baseline.
 - **Figure:** Multi-run overlay for Experiment C runs 2–5 (connections + replicas).
-

Section 6: Baseline Comparisons — Experiments D and E (Paper 2)

Length: ~900 words + figures + table **Maps directly to Section 7 of the original.**

Opening: State the comparative question directly: does performance delta arise from metric choice alone, or from controller architecture? Experiments D and E isolate these factors.

Subsection 6.1: Experiment D — HPA with Custom Connection-Count Metric

- Hypothesis: correct metric → accurate scaling but fails to preserve pods through DROP 1 (initial hypothesis — note this was *not* confirmed, and explain why).
- Setup: Prometheus Adapter with custom rule; HPA with `active_connections_per_pod`, `averageValue=100`; `stabilizationWindowSeconds=300` (HPA default); same two-cycle restorm workload; `CPU_WORK=0`; five runs.
- Results (five runs):
 - Replica targeting: accurate (8 replicas in all runs). Over-provisioning from CPU HPA eliminated.
 - Scale-up reaction: 54 ± 29 s (median 48s; range 33–104s; run 3 outlier: 104s cold-start)
 - **Pods held through DROP 1: YES** — all 8 replicas preserved in all five runs. 300s window > ~50s actual zero-connection gap.
 - CYCLE 2 transition dip: absent in Experiment D (300s window still active from CYCLE 1).

- Final scale-down: $313 \pm 96s$ (median 355s) — 2.6× longer than StatefulAutoscaler's 119s.
- Peak connections: 937 ± 43 (range 897–1,009). Pod-seconds: $3,934 \pm 148$.
- **Figure:** Multi-run overlay Experiment D (connections + replicas).
- **Key Result Box:** HPA+custom metric is necessary but not sufficient for *all* resilience properties. Three residual differences: scale-up latency (2.4×), window-boundary dip in C absent in D, final scale-down efficiency (2.6×). For gaps in 120–300s range, D holds pods while C releases — correctness depends on whether gap is transient or terminal.

Subsection 6.2: Experiment E — KEDA Baseline

- Hypothesis: KEDA cooldownPeriod=120s suppresses premature scale-down through DROP 1; lacks maxScaleDownStep rate limiting.
- Setup: KEDA v2.13.0; ScaledObject with Prometheus trigger on `sum(active_connections)`, threshold=100, minReplicaCount=2, maxReplicaCount=15, cooldownPeriod=120s; same two-cycle restorm workload; five runs.
- Results (five runs):
 - Scale to 8 replicas in all runs: mean scale-up $26 \pm 5s$ (median 27s; range 22–33s)
 - **Pods held through DROP 1:** YES — all 8 replicas in all five runs. ScaledObject status: Ready=False for ~55s but cooldown not yet expired.
 - Scale-down: never observed in five runs (cooldown not yet expired when CYCLE 2 begins).
 - Peak connections: uniformly 800 in all runs (no overshoot). Peak replicas: 8 in all runs.
 - Pod-seconds: $4,145 \pm 4$ — remarkably low variance reflects deterministic behavior when cooldown fully brackets disconnection gap.
- **Figure:** Multi-run overlay Experiment E (connections + replicas).
- **KEDA architectural distinction note:** delegation model (KEDA → HPA minReplicas → HPA executes termination). No maxScaleDownStep exposure. Once cooldown expires, scale-down converges without step-size bounding. Faster, less-bounded convergence + larger single-cycle reconnection spikes vs. StatefulAutoscaler.
- **Key Result Box:** KEDA with matched cooldownPeriod provides pod-preservation for this workload profile. Key trade-off: binary "hold-all" mechanism vs. StatefulAutoscaler's per-cycle step-rate bounding.

Subsection 6.3: Comparative Summary Across All Baselines

- **Table 4:** Quantitative comparison of all four scaler configurations (CPU HPA/B3, HPA+Custom/D, KEDA/E, StatefulAutoscaler/C) under two-cycle restorm simulation. Rows: peak replicas, scale-up reaction, pods held through DROP 1, scale-down reaction, pod-seconds, peak connections. All values mean $\pm$ std across five runs where available.

- **Figure:** Representative side-by-side comparison of Experiment C (run 4) vs. Experiment D (run 2).
 - **Sensitivity note paragraph:** Explain why reducing HPA stabilizationWindowSeconds from 300s to 60s would NOT replicate StatefulAutoscaler's DROP 1 pod-preservation. Semantic mismatch: HPA window stabilizes CPU-derived recommendations (60s of zero connections fills window with minReplicas → scale-down proceeds). StatefulAutoscaler window stabilizes connection-derived high-water marks. This is a semantic, not parametric, difference.
-

Section 7: Failure Analysis — Adversarial Scenarios (Paper 2)

Length: ~600 words + figure **Maps directly to Section 8 of the original.**

Opening: State purpose — bound the operational envelope; establish honest failure criteria.

Subsection 7.1: Failure Scenario 1 — Metric Staleness (scrape_interval=60s)

- Setup: Prometheus scrape_interval increased to 60s. Experiment C workload replayed.
- Findings: scale-up 25s, scale-down 112s, peak connections 1,082 (35.3% overshoot), peak replicas 11. 8 warm pods held through DROP 1. No connection loss.
- **Boundary condition:** If scrape_interval > scaleDownCooldownSeconds, cooldown may expire before metric recovery → premature scale-down possible. Invariant required: scaleDownCooldownSeconds > scrape_interval.

Subsection 7.2: Failure Scenario 2 — Instantaneous Connection Spike (stagger=0)

- Setup: Linear ramp removed; all 800 clients connect simultaneously.
- Findings: Scale-up reaction 6s (fastest observed). OS-level listen() backlog bottleneck during initial 2-replica absorption. Peak connections: 800 (no overshoot). Peak replicas: 8. Scale-down: 359s (stabilization window holding through extended final-drop gap).
- Note: bottleneck is Kubernetes pod scheduling, not controller decision logic. Mitigation: higher minReplicas or proactive rate-based scaling.

Subsection 7.3: Failure Scenario 3 — Prometheus Unavailability During Active Load

- Setup: `kubectl delete pod -n monitoring -l app=prometheus` during active 800-connection run. 2-minute outage.
- Findings: Controller held replicas at 8 throughout (safe-default: query failure → requeue, no action). Scale-up 21s post-recovery, scale-down 118s, peak connections 921 (15.1% overshoot from Prometheus restart lag), peak replicas 10. Zero connection loss.

- **Remaining risk:** Indefinitely long Prometheus outage → stabilization window expires → scale-down to minReplicas. Acknowledged in Limitations.

Figure: Side-by-side comparison of all three failure scenarios (failure_comparative.png from original).

Section 8: MQTT Extension — Protocol Generalization (Paper 2)

Length: ~900 words + figures + table **Maps directly to Section 10 of the original.**

Opening: State that WebSocket is the primary evaluation protocol due to instrumentation ease. MQTT is evaluated as a protocol generalization test. Same controller binary, zero source-code changes.

Infrastructure:

- Lightweight Python asyncio MQTT broker with `active_connections` on Prometheus `/metrics` endpoint
- Synthetic MQTT load generator: 1,000 client sessions, 60-second ramp
- Same Kind cluster infrastructure
- StatefulAutoscaler config: targetConnsPerPod=150, maxScaleUpStep=3, scaleDownCooldownSeconds=120, minReplicas=2

Subsection 8.1: MQTT-A — CPU HPA Baseline

- Phase 1: 339 of 1,000 clients admitted (OS-level TCP stack ceiling, not broker/controller). HPA first scales to 2 at t=68s — after connection population plateaued at 339.
- Phase 2: Manual scale to 3 replicas + 300 additional MQTT clients. Total ~639 connections, all pinned to original pod (Kubernetes Service distributes new TCP connections but does not migrate established sessions).
- Phase 3: Scale back to 1 replica → two pods removed → ~300 live MQTT sessions severed. TCP RST. Reconnection storm peak: 837 connections.
- Three MQTT-specific failure modes documented: late scale-up blindness, session pinning (no redistribution), destructive scale-down.
- **Figure:** MQTT-A plot (connections, replicas, CPU, memory, per-pod panel).

Subsection 8.2: MQTT-B — `StatefulAutoscaler` with Drain

- Phase 1: Scale-up. Controller observes `sum(active_connections)` directly → scales to 3 at t=38s. $\lceil 339/150 \rceil = 3$. **30 seconds faster than HPA (t=68s → 44% improvement).** Peak CPU: 80 millicores (never consulted).

- Phase 2: Drain-based rebalancing. Sessions pinned 176/163/0 (same transport-layer constraint as MQTT-A). Controller triggers `/drain` on hot pod at t=390s. Broker sends DISCONNECT at ~10 clients/second. Clients reconnect via Service → iptables random distribution → 247/91/0. Total connections preserved at 339 throughout drain.
- Phase 3: Forced 3→1 scale-down (mirrors MQTT-A Phase 3 for direct comparison). Controller drains sequentially: POD_C (92 connections, 50s graceful) then POD_A (already idle). All 339 connections migrate to POD_B. **Maximum single-step drop: 2** (instrument noise). Vs. MQTT-A: 639-connection cliff + 837-connection storm. Connections reach 0 only after load generator deletion.
- **Figure:** MQTT-B plot (connections, replicas, CPU, drain phases, per-pod panel).

Table 5: MQTT extension results comparison (CPU HPA/MQTT-A vs. StatefulAutoscaler/MQTT-B) — reproduce original Table 4 exactly (rows: primary signal, successful connections, first scale-up, correct replica target, per-pod redistribution, scale-down scenario, scale-down method, max single-step drop, sessions killed, peak CPU).

Key Result Box: Identical controller binary resolves all three MQTT-specific HPA failure modes. 44% faster scale-up, drain-based redistribution from 339/0/0 to 247/91/0, maximum step-drop of 2 vs. 639-connection cliff.

Section 9: Discussion (Paper 2)

Length: ~600 words **Draws from the original Discussion section. Adapts subsections to Paper 2's solution framing.**

Subsection 9.1: The Correct Signal Changes Everything

- Experiment C operated at CPU_WORK=0 throughout. A controller on the correct signal (connections) outperforms a proportional controller on the wrong signal (CPU) categorically, not just marginally.
- Philosophical implication: architecture (signal choice + lifecycle semantics) matters more than algorithm sophistication.

Subsection 9.2: Stabilization Semantics as a First-Class Design Dimension

- Revisit the semantic distinction: HPA stabilizes CPU-derived replica recommendations; StatefulAutoscaler stabilizes connection-derived desired counts.
- This distinction explains why Experiment D (correct metric, HPA stabilization) succeeds for the 90s DROP 1 but fails at efficiency: HPA's 300s window is semantically "dumb" — it holds pods not because it knows connections are present but because 300s haven't elapsed. StatefulAutoscaler holds pods because it knows connections *were* present.

- Implication: for disconnection gaps > stabilizationWindowSeconds (Experiment D would fail at gaps > 300s; StatefulAutoscaler would fail at gaps > 120s), controller-architecture-level stabilization must be tuned to the expected disconnection gap distribution.

Subsection 9.3: Step-Rate Limiting as a Safety Mechanism

- maxScaleUpStep and maxScaleDownStep: contribute to monotonic trajectories and avoid explosive scaling events.
- Particularly important: scale-down step limiting bounds the "blast radius" — maximum connections destroyed by any single scale-down event = maxScaleDownStep × targetConnsPerPod.
- Contrast with KEDA: no per-cycle step limit; once cooldown expires, HPA may remove many pods in a single reconciliation.

Subsection 9.4: Generalizability Beyond WebSocket

- The sole requirement for applying this controller to any protocol: a Prometheus gauge reflecting current active connection population.
- Direct applicability: gRPC streaming, MQTT (demonstrated), Server-Sent Events, database connection pool managers.
- MQTT validation confirms protocol-agnosticism. The `active_connections` abstraction is protocol-neutral.

Subsection 9.5: Operational and Fairness Considerations

- Connection-count scaling implicitly couples resource allocation to client session behavior.
- Multi-tenant risk: high-connection users driving disproportionate scale-up.
- Mitigation: namespace-scoped resource quotas, per-user connection limits at application layer.

Section 10: Limitations (Paper 2)

Length: ~500 words **Include** all limitations from the original that pertain to the StatefulAutoscaler design and evaluation. **Do NOT** repeat Paper 1's limitations (evaluation environment, workload generalizability for the HPA characterization).

Include (adapted from original Section 12):

1. **Evaluation environment:** Single-machine Kind cluster; absolute timing values may differ on multi-node cloud infrastructure.

2. **Workload generalizability:** Synthetic ramps and deterministic gaps; real-world bursty traffic not yet validated. For stationary Poisson arrivals and gradual long-term drift: bounded and predictable degradation modes described.
 3. **Observability dependency:** `scaleDownCooldownSeconds > scrape_interval` invariant must be maintained. Indefinitely long Prometheus outage → window expiry → scale-down to `minReplicas`.
 4. **No predictive scaling:** Reactive only. Rate-based proactive signal $((\text{rate}(\text{new_connections_total}[1\text{m}])))$ is outside current scope.
 5. **Drain semantics scope:** Validated in MQTT-B only; WebSocket drain not yet tested end-to-end.
 6. **Volatile stabilization state:** Sliding window in process memory; controller restart resets history. Persist to CR status subresource to eliminate risk.
 7. **Window boundary edge case:** CYCLE 2 transient dip to 5-6 replicas; configurable minimum hold-time following last scale-up would prevent this.
 8. **Per-pod granularity:** WebSocket evaluation uses aggregate $(\text{sum}(\text{active_connections}))$. Hot-pod imbalance detection requires per-pod queries (demonstrated in MQTT-B; WebSocket extension planned).
 9. **MQTT session redistribution mechanism:** Drain-reconnect requires client-side retry logic. Broker-level clustering would be more transparent.
 10. **MQTT connection-ramp bottleneck:** 339/1,000 clients admitted — OS-level TCP ceiling, not controller limitation. Pre-warming or retry-capable clients required.
 11. **MQTT memory instrumentation gap:** `memory.log` collector recorded zeros; `cpu.log` samples showed ~79Mi peak. Qualitative CPU-independence result unaffected; memory conclusions are exploratory.
 12. **MQTT multi-run replication:** MQTT-A and MQTT-B are single runs. Five-run statistical replication (as for Experiments C-E) is future work.
-

Section 11: Future Work (Paper 2)

Length: ~600 words **Maps directly to Section 11 of the original. Include all subsections.**

Subsection 11.1: Overcoming the TCP/OS-Level Connection Ceiling

- Root cause explanation: Linux ephemeral port range, `listen()` backlog, `net.core.somaxconn`, Docker-in-Docker networking in Kind.
- Five mitigation strategies (verbatim from original):
 - Kernel parameter tuning (`somaxconn`, `tcp_max_syn_backlog`, `SO_BACKLOG`)
 - Client-side retry with exponential backoff
 - Connection distribution via headless service or per-pod addressing

- Proactive pre-warming (higher minReplicas or rate-based trigger)
- Real cluster vs. Kind evaluation

Subsection 11.2: MQTT — Future Optimizations

- Broker-native clustering for seamless session migration (EMQX, VerneMQ, HiveMQ)
- QoS-aware scaling signals (QoS 2 sessions cost more; weight targetConnsPerPod)
- Persistent session management (cleanSession=false; external session store like Redis)
- Last-Will message handling during drain (suppress LWT on graceful DISCONNECT)
- Multi-run statistical replication for MQTT-A and MQTT-B

Subsection 11.3: WebSocket — Future Extensions

- Predictive and proactive scaling (rate(new_connections_total[1m]) as leading indicator)
 - Drain semantics for WebSocket (protocol-level close frame + reconnect hint)
 - Stabilization window persistence (CR status subresource or etcd)
 - Per-pod granularity for imbalance detection (hot-pod drain trigger)
 - Real-world traffic trace validation
 - Multi-cluster and federated scaling
-

Section 12: Conclusion (Paper 2)

Length: ~300 words **Must contain:**

- Restate the paper's scope: design, implementation, and experimental validation of the `StatefulAutoscaler`.
- Brief motivating context (two sentences, citing Paper 1): three documented HPA failure modes (87.5% over-provisioning, 1,400 conn/s storms, 800→79 permanent destruction) established in the companion paper.
- Experiment C results (cite five-run statistics): exact replica targeting, zero connection loss, 8 pods through 90s gap, scale-down $119 \pm 2s$.
- Baseline comparison nuance: HPA+custom metric (D) and KEDA (E) both preserve pods through DROP 1 with correct configuration; StatefulAutoscaler differentiates on scale-up latency (22s vs. 54s vs. 26s) and final scale-down efficiency (119s vs. 313s).
- Adversarial envelope: graceful degradation under metric staleness and Prometheus outage; scheduling latency as binding constraint for instantaneous spikes.
- MQTT generalization: identical binary, 44% faster scale-up, 0 vs. ~300 sessions destroyed, drain-reconnect achieving redistribution without application changes.

- Closing statement: as cloud-native architectures adopt persistent-connection protocols (WebSocket, gRPC, MQTT, SSE), connection-aware autoscaling with protocol-appropriate drain semantics is an increasingly critical infrastructure requirement. The `StatefulAutoscaler` demonstrates that a controller of modest implementation complexity can eliminate the fundamental failure modes of CPU-based HPA for this workload class.
-

Data Availability (Paper 2)

Point to: `https://github.com/MistaHolmes/ws-k8-controller` Describe: StatefulAutoscaler controller source, CRD manifests, all experiment scripts (C, D, E, MQTT-A, MQTT-B), multi-run scripts, raw and processed results, analysis scripts, replication instructions in Replication.md.

Paper 2 — Section Order Summary

#	Section Title	~Word Count
—	Abstract	300
1	Introduction	600
2	Related Work	550
3	Background	300
4	Controller Design and Implementation	1,000
5	Experimental Evaluation — Experiment C	1,100
6	Baseline Comparisons — Experiments D and E	900
7	Failure Analysis — Adversarial Scenarios	600
8	MQTT Extension	900
9	Discussion	600
10	Limitations	500
11	Future Work	600
12	Conclusion	300

#	Section Title	~Word Count
—	Data Availability + Acknowledgements	100
Total		~7,350

5. Cross-Paper Dependency Map

This table governs how each paper references the other to maintain independence without redundancy.

Content	Paper 1 Treatment	Paper 2 Treatment
HPA proportional control law (Eq. 1)	Defines and uses it	Cites Paper 1; reproduces equation for self-containment
KRM/PCM baseline experiments	Full description and figures	One sentence + citation to Paper 1
Experiments A, B1, B2, B3	Full description and figures	1-paragraph summary with 3 key numbers; full citation
Reconnection storm definition	Defines it	Uses the definition; cites Paper 1
30-second SIGKILL limbo	Full characterization	Referenced briefly in Section 2 Related Work; cites Paper 1
StatefulAutoscaler	Not mentioned	Full design + evaluation
Experiments C, D, E	Not mentioned	Full description and figures
MQTT extension	Not mentioned	Full description and figures
Adversarial scenarios	Not mentioned	Full description and figures
HPA architecture figure	Owens it	Reproduces with citation to nguyen2020hpa (not to Paper 1)

Citation convention: Paper 2 refers to Paper 1 as "[Paper 1 authors], 'CPU-Based Autoscaling is Harmful...', [journal], [year]" in the reference list. Before Paper 1 is published, Paper 2 can cite it as "[under review]" or as a preprint.

6. Figures and Tables Assignment

Figures — Paper 1

Figure	Original Label	Content	Source File
1	fig:hpa_arch	HPA feedback loop architecture (from nguyen2020hpa)	K8-hpa-architecuture.png
2	fig:krm_cpu	KRM baseline: CPU over time	krm-results/cpu_over_time.png
3	fig:krm_desired	KRM baseline: desired vs. current replicas	krm-results/desired_vs_current.png
4	fig:pcm_scrape	PCM baseline: scrape interval comparison	pcm-results/PCM-scraping_period_comparison.png
5	fig:exp_a_replicas	Experiment A: replica count timeline	experiment-a-hpa/replicas.png
6	fig:exp_b1_replicas	Experiment B1: saturation at maxReplicas	experiment-b1-hpa-churn/replicas.png
7	fig:exp_b2_reconnections	Experiment B2: reconnection rate	experiment-b2-hpa-churn-instrumented/reconnections.png
8	fig:exp_b2_connections	Experiment B2: connection count + overshoot	experiment-b2-hpa-churn-instrumented/connections.png
9	fig:exp_b2_scaling	Experiment B2: replica changes vs. connections	experiment-b2-hpa-churn-instrumented/scaling_activity.png
10	fig:exp_b3_connections	Experiment B3: staircase connection destruction	experiment-b3-hpa-idle-connections/connections.png
11	fig:exp_b3_combined	Experiment B3: causal proof (CPU→replicas→connections)	experiment-b3-hpa-idle-connections/combined.png

Tables — Paper 1

Table	Original Label	Content
1	tab:failure_summary	Progressive evidence chain summary (4 rows)

Figures — Paper 2

Figure	Original Label	Content	Source File
1	fig:hpa_arch	HPA architecture (background, same as P1 Fig 1)	K8-hpa-architecuture.png
2	fig:system_arch	StatefulAutoscaler system architecture + CRD	StatefulAutoScaler- System_Architecture-CRD.png
3	fig:reconcile_uml	UML activity diagram of reconciliation loop	StatefulAutoscaler Reconciliation Flow-UML.png
4	fig:exp_c_connections	Experiment C: connections over timeline	experiment-c-stateful/connections.png
5	fig:exp_c_replicas	Experiment C: replica count (flat bridge)	experiment-c-stateful/replicas.png
6	fig:exp_c_combined	Experiment C: combined signal view	experiment-c-stateful/combined.png
7	fig:exp_c_multi	Experiment C: multi-run overlay (runs 2-5)	experiment-c- multi/multi_timeseries.png
8	fig:exp_d_multi	Experiment D: multi-run overlay (5 runs)	experiment-d- multi/multi_timeseries.png
9	fig:exp_e_multi	Experiment E: multi-run overlay (5 runs)	experiment-e- keda/multi_timeseries.png
10	fig:comparison_c_vs_d	Side-by-side C vs. D representative runs	comparison-c- d/comparison_c_vs_d.png
11	fig:failure_comparative	Three failure scenarios side-by-side	failure- scenarios/failure_comparative.png

Figure	Original Label	Content	Source File
12	fig:mqtt_a_plot	MQTT-A: CPU HPA baseline	experiment-a-hpa/plot.png
13	fig:mqtt_b_plot	MQTT-B: StatefulAutoscaler with drain	experiment-b-stateful/plot.png

Tables — Paper 2

Table	Original Label	Content
1	tab:crd	CRD specification fields
2	tab:exp_c_setup	Experiment C vs. B3 parametric comparison
3	tab:head_to_head	Head-to-head: HPA/B3 vs. StatefulAutoscaler/C
4	tab:baseline_compare	All four scalers comparative summary (5-run stats)
5	tab:mqtt_results	MQTT: CPU HPA vs. StatefulAutoscaler with drain

7. Equations Assignment

Equation	Content	Paper 1	Paper 2
Eq. 1 (HPA control law)	$\text{desiredReplicas} = \lceil \frac{\text{currentReplicas} \times (\text{currentMetric} / \text{targetMetric})}{1} \rceil$	Defines	Reproduces with citation
Eq. 2 (StatefulAutoscaler)	$\text{desiredReplicas} = \lceil \frac{\text{sum}(\text{active_connections})}{T} \rceil$	Not present	Defines
Eq. 3 (Convergence bound)	$\lceil \frac{[(\text{current} - \text{desired}) / \text{maxScaleDownStep}]}{\text{cycles}} \rceil$	Not present	Defines

References Used in Both Papers (must appear in both reference lists)

- burns2016borg, verma2015large (Kubernetes motivation)
- k8s_hpa_docs (HPA documentation)
- nguyen2020hpa (HPA architecture — cited frequently in both)
- websocket_rfc (WebSocket protocol)
- dickel2019stateful (stateful autoscaling — cited in related work of both)
- casalicchio2019auto, lorido2014auto (HPA behavior surveys)
- kind_docs (Kind cluster)
- prometheus_adapter

References Used Only in Paper 1

- qu2018auto, rossi2020horizontal (HTTP-rate autoscaling — cited in Related Work P1)

References Used Only in Paper 2

- kubebuilder_docs (Kubebuilder SDK — only needed for controller implementation)
- keda_docs (KEDA — only needed for Experiment E)
- imdoukh2020machine, toka2021machine, zhong2020machine (ML autoscaling — extended related work in P2)
- soni2017mqtt (MQTT protocol characteristics — MQTT section)
- mqtt_v31 (MQTT specification — conclusion of P2)

New Reference Needed in Paper 2

- **[Paper 1]** — cite the companion paper. Before publication: "[under review at same journal]". After publication: full citation.
-

9. Abstract Drafts

Paper 1 Abstract Draft

The Kubernetes Horizontal Pod Autoscaler (HPA) implements a proportional control law over CPU utilization as its primary scaling signal — an architecture that performs reliably for stateless, request-driven workloads but is structurally mismatched to stateful, persistent-connection protocols such as WebSocket. In WebSocket deployments, the primary unit of

computational commitment is an active network connection whose session state persists independently of CPU activity; the termination of any pod immediately and irrecoverably severs every connection it hosts. This paper presents a systematic empirical characterization of HPA failure under WebSocket workloads through a progressive chain of four controlled experiments. Experiment A establishes a correct-behavior baseline under the idealized condition of CPU-to-connection proportionality. Experiment B1 demonstrates that HPA over-provisions by 87.5% under cyclic load, exhausting `maxReplicas` within 99 seconds and requiring 650 seconds to recover. Experiment B2-Instrumented uses Prometheus instrumentation to measure reconnection storm rates causally linked to HPA scale-down decisions, with a peak observed rate of 1,400 connections/second and a 51.9% connection count overshoot (1,215 vs. 800 target). Experiment B3 demonstrates permanent, irrecoverable staircase connection destruction under non-reconnecting clients, with the active session population monotonically reduced from 800 to 79 with zero recovery. We establish that these failure modes are architectural rather than configurational: they arise from the semantic invalidity of CPU as a scaling signal for persistent-connection workloads and cannot be remediated by HPA parameter tuning alone. The findings motivate the design of connection-aware autoscaling policies as presented in the companion paper.

Keywords: Kubernetes, Horizontal Pod Autoscaler, WebSocket, Stateful Workloads, Reconnection Storm, Prometheus, Measurement Study, Connection-Aware Autoscaling

Paper 2 Abstract Draft

CPU-based Kubernetes HPA is empirically shown in the companion paper to exhibit three structural failure modes for stateful WebSocket workloads: 87.5% over-provisioning, reconnection storms reaching 1,400 connections/second, and permanent staircase connection destruction. This paper presents the design, implementation, and multi-baseline experimental validation of the `StatefulAutoscaler` — a custom Kubernetes controller built on the Kubebuilder SDK that resolves all three failure modes by scaling exclusively on `sum(active_connections)` queried from Prometheus and incorporating a sliding-window stabilization mechanism (`scaleDownCooldownSeconds`) to suppress premature scale-down during transient connection drops. Controlled experimental evaluation across a two-cycle restorm simulation (Experiment C, five independent runs) demonstrates exact replica targeting ($\lceil 800/100 \rceil = 8$ replicas, 0% waste), zero connection loss across all observed scaling events, and complete pod preservation through a 90-second disconnection gap via the stabilization window (scale-down reaction time 119 ± 2 s; pod-seconds $4,280 \pm 67$). Comparative baseline evaluation against HPA augmented with a custom connection-count metric via the Prometheus Adapter (Experiment D, five runs) and KEDA configured with a matching `cooldownPeriod` (Experiment E, five runs) reveals a nuanced finding: all three connection-aware configurations preserve pods through the 90-second DROP 1 gap, with the `StatefulAutoscaler` differentiating on scale-up latency (22s vs. 54s for Experiment D) and final scale-down efficiency (119s vs. 313s). Three

adversarial failure scenarios establish the operational envelope: metric staleness degrades gracefully to a reaction lag bounded by the scrape interval; instantaneous connection spikes expose pod scheduling latency as the binding constraint; Prometheus unavailability triggers safe-default hold behavior with no connection loss. MQTT generalization experiments confirm protocol-agnosticism: the identical controller binary, with zero source-code changes, scales 44% faster than CPU HPA (38s vs. 68s), preserves all 339 sessions during a forced scale-down (maximum step-drop: 2 vs. 639-connection cliff), and redistributes sessions across pods via drain-reconnect semantics.

Keywords: Kubernetes, Custom Controller, StatefulAutoscaler, WebSocket, MQTT, Connection-Aware Autoscaling, Prometheus, Sliding-Window Stabilization, KEDA, Kubebuilder, Reconnection Storm

10. Key Writing Guidelines for Both Papers

Tone and Framing

- Paper 1 is a **measurement paper**: every claim must be backed by a figure, table, or explicit quantitative result. Do not speculate about solutions.
- Paper 2 is a **systems paper**: design decisions must be explicitly justified (why this parameter, why this mechanism). Evaluation must be rigorous with five-run statistics.

Handling the Companion Paper Relationship

- Each paper must be fully self-contained — a reviewer who has not read the companion paper must be able to evaluate it independently.
- Paper 1 ends with a forward pointer to Paper 2 but does not describe the `StatefulAutoscaler` or its results.
- Paper 2 treats Paper 1's results as established prior work with a 1-paragraph summary and full citation. It does not reproduce the HPA failure experiments in detail.

Statistical Reporting Standards

- All multi-run results: report as `(mean ± std)` with median and range in parentheses.
- Single-run results (B3, MQTT-A, MQTT-B): explicitly note that only a single run was conducted.
- Run 1 of Experiment C: explicitly excluded with documented reason (persistent-replica test harness artefact). Statistics computed over runs 2–5 only.

Figures

- All figures referenced in-text before they appear (or at point of reference for two-column format).
- All `(figure*)` (full-width) figures: Experiments B2/B3 combined views, system architecture, failure comparative, MQTT plots.
- All `(figure)` (single-column) figures: replica count timelines, multi-run overlays, UML diagram.
- Caption format: include experiment name, what the key visual feature shows, and what conclusion it supports.

Key Result Boxes

- Both papers use the `(keyresult)` tcolorbox environment for highlighted conclusions at the end of each experiment subsection.
- Paper 1: 4 key result boxes (one per experiment: A, B1, B2, B3).
- Paper 2: 5 key result boxes (Experiment C, Experiment D, Experiment E, MQTT, plus one for adversarial summary or failure analysis).

Numbers to Never Get Wrong

The following numbers appear in both papers and must be consistent:

- Over-provisioning: **87.5%** (15 vs. 8 replicas); maxReplicas reached in **99 seconds**
 - Peak reconnection storm rate: **1,400 connections/second** (Cycle 1: 1,400.9 conn/s)
 - Connection overshoot in B2: **1,215 connections** (51.9% above 800-target)
 - B3 destruction: **800→744→697→445→79** (staircase values)
 - Experiment C target: **[800/100] = 8 replicas**
 - Experiment C five-run stats: scale-up **22 ± 6s**, scale-down **119 ± 2s**, pod-seconds **4,280 ± 67**
 - Experiment D: scale-up **54 ± 29s**, scale-down **313 ± 96s**
 - Experiment E: scale-up **26 ± 5s**, pod-seconds **4,145 ± 4**
 - MQTT-B: scale-up at **t=38s** (vs. t=68s for MQTT-A); max step-drop **2**; sessions preserved **339**
 - Failure Scenario 1: peak connections **1,082** (35.3% overshoot), peak replicas **11**
 - Failure Scenario 2: scale-up reaction **6s**
 - Failure Scenario 3: peak connections **921** (15.1% overshoot), peak replicas **10**
-

11. Risk Flags and Edge Cases

Risk 1: The "flat bridge" narrative

The flat bridge during DROP 1 in Experiment C is the central empirical result of Paper 2. It must be described with absolute precision: replicas held at **exactly 8** for the **full 90-second gap**, while connection count was **exactly 0**. Do not soften this or introduce ambiguity. The contrast with B3 (where connections are destroyed by this point) must be made explicit.

Risk 2: Experiment D's result was surprising

The initial hypothesis for Experiment D was that HPA with a custom metric would *fail* to preserve pods through DROP 1. It did not fail — it preserved pods via the 300-second stabilization window. The paper must present this honestly as a finding that *refines* the narrative: metric choice alone is not the differentiator for the 90-second DROP 1 scenario; the differentiators are scale-up latency and final scale-down efficiency. Do not hide or minimize this.

Risk 3: Run 1 of Experiment C

Run 1 is explicitly excluded from all statistics with a documented reason. This must be mentioned in a dedicated paragraph (as in the original) and not swept under the rug. The exclusion reason (persistent-replica test harness artefact, not controller behavior) must be stated clearly.

Risk 4: MQTT ceiling at 339/1,000

Both MQTT-A and MQTT-B admitted 339 of 1,000 clients. This is not a controller failure — it is an OS-level TCP stack limitation in the Kind environment. This must be clearly attributed and not presented as a limitation of the `StatefulAutoscaler`. The Future Work section (Subsection 11.1) addresses it directly.

Risk 5: Window boundary transient dip in Experiment C

During early CYCLE 2, a brief dip to 5–6 replicas was observed. This is a real behavior of the controller that must be documented honestly (as in the original), not omitted. It is mentioned in Experiment C (Subsection 5.5), acknowledged in Limitations (item 7), and addressed in Future Work. The key point: no connection loss occurred during the dip.

Risk 6: MQTT memory instrumentation gap

The MQTT-B memory.log recorded zeros. This must be noted in Limitations (item 11) as an instrumentation defect, and MQTT memory conclusions must be labeled as exploratory. The cpu.log values (~79Mi observed via `kubectl top`) should be cited for completeness without overstating their reliability.

Risk 7: Paper 2 Section ordering

The original paper has adversarial scenarios after baselines, MQTT after adversarial. This ordering is preserved in Paper 2. Do not reorder sections without good reason — the logical flow (design → primary validation → comparative baselines → failure bounds → generalization) is intentional.

Risk 8: Dual GitHub repositories

Paper 1 cites <https://github.com/AbhasBenevolentDictator/STAR> (HPA experiments A, B1, B2, B3). Paper 2 cites <https://github.com/MistaHolmes/ws-k8-controller> (StatefulAutoscaler + Experiments C, D, E, MQTT). These must not be mixed up. Both must be verified as publicly accessible before submission.

End of Proposal Document

Prepared for handoff — June 2026 All content derived exclusively from the source paper. No new experiments, data, or results introduced.